

Department of Computer Science and Engineering

Dhulikhel, Kavre



**Project Report on
Comparative Evaluation of Machine Learning
Classification Algorithms for Loan Eligibility Prediction**

Submitted By

Mohit Pokharel

Sabil Shrestha

Submitted To

Prof. Bal Krishna Bal

Submission Date

7th September 2026

Abstract

Financial institutions process many loan applications from applicants with different financial and demographic backgrounds. Traditional manual approval can be time-consuming and may produce inconsistent decisions. Machine learning can help automate this process by identifying patterns in historical loan applications.

In this project, loan eligibility prediction is treated as a **binary classification problem** using **614 loan applications with 13 original features**. The workflow includes data cleaning, exploratory analysis, feature engineering, missing-value treatment, categorical encoding, scaling, cross-validation, and model evaluation.

The following **seven machine learning algorithms** were compared:

- K-Nearest Neighbors (KNN)
- Decision Tree
- Random Forest
- Logistic Regression
- Support Vector Machine (SVM)
- Gaussian Naive Bayes
- Histogram-based Gradient Boosting

Logistic Regression achieved the best cross-validation F1 score (**0.868**), while Random Forest achieved the highest classical ROC-AUC (**0.787**). A neural network and PCA-based Logistic Regression were also evaluated, with PCA Logistic Regression achieving **0.862 accuracy, 0.908 F1, and 0.884 ROC-AUC**.

The primary research question is:

Which machine learning algorithm and preprocessing approach provides the most accurate and reliable loan-approval predictions on this dataset?

Notebook: [ML Mini Project](#)

1. Introduction

Loan approval is a binary decision problem in which an applicant is classified as eligible or ineligible according to information available to the lending institution. A machine-learning model can learn patterns in historical applications and provide a consistent prediction for new applicants. Such a system can support analysts by prioritizing applications, identifying cases requiring review, and providing an empirical basis for comparing applicant profiles.

The supplied notebook frames the task as supervised binary classification. The target variable, `Loan_Status`, originally contains Y for approved applications and N for denied applications. The dataset has 614 observations. Of these, 422 applications are approved (68.73%) and 192 are denied (31.27%). The class distribution is therefore moderately imbalanced toward approval, making F1 score and ROC-AUC useful complements to raw accuracy.

1.1 Problem Statement

The central problem is to develop and compare predictive models capable of estimating loan approval status from applicant and loan characteristics. The study is not limited to one algorithm. Instead, it evaluates several model families with different assumptions and learning mechanisms, then extends the comparison to a neural network and a dimensionality-reduction experiment.

1.2 Objectives

- Prepare the loan application dataset for machine-learning analysis while preserving the information contained in its original variables.
- Investigate missingness, target distribution, numeric feature distributions, credit history, property area, and correlations.
- Engineer economically meaningful variables, particularly total income and the loan-to-income ratio.
- Build preprocessing pipelines that separately handle numeric and categorical variables.
- Compare seven classical classifiers using stratified five-fold cross-validation.
- Evaluate the trained models on a held-out 20% test set using Accuracy, F1, and ROC-AUC.
- Assess whether a neural network can improve predictive performance.
- Use PCA to investigate whether a lower-dimensional feature representation can retain useful predictive information.

1.3 Scope

The scope is limited to the variables and experiments implemented in the supplied notebook. No external dataset, external model, or additional hyperparameter-search results are assumed. Conclusions therefore describe the experimental evidence present in the notebook rather than making claims about all possible lending datasets or production environments.

2. Dataset and Exploratory Data Analysis

2.1 Dataset Description

The dataset contains 614 observations and 13 original columns. The fields are Loan_ID, Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area, and Loan_Status. Eight fields are object/string variables, four are floating-point variables, and ApplicantIncome is an integer variable.

Variable group	Columns / description
Identification	Loan_ID — removed before modeling
Demographic	Gender, Married, Dependents, Education, Self_Employed
Financial	ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term
Credit	Credit_History
Property	Property_Area
Target	Loan_Status: Y/N, later encoded as 1/0

2.2 Missing Values

Missing values occur in several predictors. Credit_History has the largest amount of missing data, with 50 missing values (8.1% of the dataset). Self_Employed has 32 missing values (5.2%), LoanAmount has 22 (3.6%), Dependents has 15 (2.4%), Loan_Amount_Term has 14 (2.3%), Gender has 13 (2.1%), and Married has 3 (0.5%). Loan_ID, Education, Property_Area, and Loan_Status have no missing values.

Feature	Missing	Approx. %
Credit_History	50	8.1%
Self_Employed	32	5.2%
LoanAmount	22	3.6%
Dependents	15	2.4%
Loan_Amount_Term	14	2.3%
Gender	13	2.1%
Married	3	0.5%

2.3 Exploratory Findings

The notebook visualizes the target distribution, missing-value counts, histograms for the main numeric variables, approval rate by credit history, approval rate by property area, and a correlation heatmap. The credit-history analysis is particularly relevant because it directly compares approval rates across the available credit-history categories. Property area is also examined to determine whether approval rates differ between rural, semiurban, and urban applicants.

The numeric histograms are used to inspect ApplicantIncome, CoapplicantIncome, LoanAmount, and Loan_Amount_Term before modeling. This is important because financial variables can have different scales and distributions. The correlation heatmap provides an additional view of linear relationships among the numeric predictors and Credit_History.

3. Methodology

3.1 Data Cleaning and Feature Preparation

The notebook first copies the original dataframe and removes Loan_ID because it is an identifier rather than a meaningful predictive attribute. Loan_Status is mapped from Y/N to 1/0. The Dependents variable contains the category 3+, which is converted to 3 so that the variable can be treated consistently as an ordered categorical/numeric-like feature during preprocessing.

3.2 Feature Engineering

Two additional variables are created. Total_Income is defined as ApplicantIncome plus CoapplicantIncome. This combines the applicant's and coapplicant's income into a single measure of household/application-level earning capacity. Loan_Income_Ratio is defined as LoanAmount divided by Total_Income. The implementation replaces zero total income with a missing value before division, avoiding division-by-zero problems.

Feature	Construction	Purpose
Total_Income	ApplicantIncome + CoapplicantIncome	Represent combined income capacity
Loan_Income_Ratio	LoanAmount / Total_Income	Represent loan size relative to income

After feature engineering, the feature matrix contains 13 predictors: the original non-target variables after dropping Loan_ID, plus Total_Income and Loan_Income_Ratio. The target contains 614 binary observations.

3.3 Preprocessing

The notebook uses scikit-learn Pipeline and ColumnTransformer objects to keep preprocessing attached to each model. Numeric variables are imputed with the median. For models requiring scale-sensitive inputs, numeric variables are additionally standardized using StandardScaler. Categorical variables are imputed with the most frequent category and then one-hot encoded with unknown categories ignored.

The scaled numeric set contains ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Total_Income, Loan_Income_Ratio, and Credit_History. The categorical set contains Gender, Married, Dependents, Education, Self_Employed, and Property_Area. The processed representation used by the neural network and PCA contains 22 features.

3.4 Train-Test Strategy

The data are divided into 80% training and 20% testing using `train_test_split` with stratification and `random_state=42`. This produces 491 training observations and 123 test observations. The training set contains approximately 68.64% approved and 31.36% denied applications; the test set contains approximately 69.11% approved and 30.89% denied applications. Stratification helps preserve the target distribution between the two subsets.

4. Experimental Design and Models

4.1 Classical Machine-Learning Models

Seven models are evaluated. Each model is embedded in a preprocessing pipeline so that missing-value handling, encoding, and scaling are performed consistently within the modeling workflow.

Model	Main characteristic	Preprocessing
SVM (RBF)	Non-linear kernel classifier; probability enabled	Scaled numeric + one-hot categorical
Logistic Regression	Linear probabilistic classifier	Scaled numeric + one-hot categorical
Random Forest	Ensemble of decision trees	Unscaled numeric + one-hot categorical
KNN	Distance-based neighborhood classifier	Scaled numeric + one-hot categorical
HistGradientBoosting	Gradient-boosted tree model	Unscaled numeric + one-hot categorical
Decision Tree	Single tree classifier	Unscaled numeric + one-hot categorical
Naive Bayes	Gaussian probabilistic classifier	Scaled numeric + one-hot categorical

4.2 Cross-Validation

Five-fold StratifiedKfold cross-validation is applied to the training set. Each fold maintains the class proportions as closely as possible. Three metrics are collected: Accuracy, F1 score, and ROC-AUC. The notebook sorts the classical models primarily by mean cross-validation F1 score.

Model	CV Accuracy	CV F1	CV ROC-AUC
Logistic Regression	0.796	0.868	0.731
SVM (RBF)	0.792	0.866	0.746
Naive Bayes	0.772	0.848	0.734
Random Forest	0.772	0.848	0.787
KNN	0.762	0.842	0.703
HistGradientBoosting	0.762	0.837	0.741
Decision Tree	0.711	0.791	0.659

4.3 Evaluation Metrics

Accuracy measures the proportion of correct predictions. F1 score is the harmonic mean of precision and recall and is particularly informative when the classes are not perfectly balanced. ROC-AUC measures the model's ability to rank positive cases above negative cases over different classification thresholds. In this project, the positive class is loan approval.

The use of several metrics is important because the dataset contains more approvals than denials. A model could achieve a reasonable accuracy by favoring the majority class, while a lower F1 or ROC-AUC may reveal weaker performance on the decision boundary or ranking task.

5. Results

5.1 Classical Model Cross-Validation Results

Logistic Regression achieved the highest cross-validation F1 score at 0.868, followed very closely by the RBF SVM at 0.866. Their cross-validation accuracies were 0.796 and 0.792, respectively. The difference in F1 is small, indicating that the two approaches produced broadly similar classification quality under the selected preprocessing and default model settings.

Random Forest deserves special attention because its cross-validation ROC-AUC of 0.787 was the highest among the seven classical models. This indicates that, although its F1 score was below Logistic Regression and SVM, it produced the strongest ranking discrimination in the cross-validation experiment. Decision Tree was the weakest classical model on all three reported cross-validation measures, with 0.711 accuracy, 0.791 F1, and 0.659 ROC-AUC.

5.2 Neural Network Results

The neural network uses 22 processed input features. Its architecture consists of a 64-unit dense layer with ReLU activation, batch normalization, 30% dropout, a 32-unit ReLU layer, 20% dropout, and a one-unit sigmoid output. Adam with a learning rate of 0.001 is used with binary cross-entropy loss. Training allows up to 150 epochs but

applies early stopping with patience 15 and restores the best weights. The best validation-loss epoch recorded by the notebook was epoch 10.

Metric	Neural Network Test Result
Accuracy	0.862
F1 Score	0.906
ROC-AUC	0.867
Denied precision	0.89
Denied recall	0.63
Denied F1	0.74
Approved precision	0.85
Approved recall	0.96
Approved F1	0.91

The neural network achieved 0.862 test accuracy and a 0.906 F1 score. Its ROC-AUC was 0.867. The classification report shows stronger recall for approved applications (0.96) than for denied applications (0.63). Thus, the model is particularly effective at identifying applications that belong to the approval class, while a larger fraction of denied cases are predicted as approved.

5.3 PCA Results

PCA was fitted to the same scaled/encoded representation used by the neural network. The notebook reports that 10 principal components are sufficient to explain at least 90% of the variance, reducing the representation from 22 features to 10 components.

Experiment	Accuracy	F1	ROC-AUC
Logistic Regression + PCA (10 components)	0.862	0.908	0.884
Logistic Regression, full feature set	0.862	0.908	0.875

The PCA experiment is notable because accuracy and F1 remain unchanged while ROC-AUC increases from 0.875 to 0.884. In the notebook's reported results, dimensionality reduction therefore preserves the classification outcome while improving the ranking metric.

6. Limitations

Small dataset size. The dataset contains only 614 loan applications, with 491 observations used for training and 123 for final testing. This is relatively small for a predictive lending problem and means that performance estimates can be sensitive to the particular train-test split. A larger and more diverse dataset would provide more reliable estimates of generalization.

Class imbalance. The target contains 422 approved applications and 192 denied applications, so approvals represent the majority class. Although stratified splitting and F1/ROC-AUC are used, the project does not apply class weighting, resampling, or a dedicated cost-sensitive learning strategy. Consequently, the reported models may still favor the approval class.

Limited hyperparameter optimization. The notebook compares several algorithms but does not perform a systematic grid search, randomized search, or Bayesian optimization for their hyperparameters. Therefore, the results represent the selected model configurations rather than the maximum performance that each algorithm could potentially achieve.

Different validation procedures across models. The seven classical models are evaluated using five-fold stratified cross-validation, whereas the neural network is trained once with an internal validation split and evaluated on the held-out test set. PCA plus Logistic Regression is also evaluated on the test set without the same cross-validation procedure. Because the evaluation protocols differ, their reported scores should not be interpreted as a perfectly controlled head-to-head comparison.

Single train-test split. The final test results are based on one 80/20 split with `random_state=42`. While stratification improves reproducibility and preserves the class ratio, a single split cannot show how stable the results are across alternative samples. Repeated cross-validation or nested cross-validation would provide stronger evidence.

Limited feature engineering. Only `Total_Income` and `Loan_Income_Ratio` are engineered beyond the original variables. Other potentially informative relationships, interactions, transformations, or domain-specific indicators are not explored. This may limit the ability of the models to capture more complex applicant patterns.

PCA preserves variance, not necessarily predictive information. The PCA experiment selects 10 components because they retain at least 90% of the input variance. High explained variance does not guarantee that the retained components contain the most useful information for loan approval prediction. PCA is unsupervised, so it does not use the target variable when deciding which directions to preserve.

Limited interpretability analysis. The project focuses primarily on predictive metrics. It does not include detailed feature-importance analysis, model explanations, calibration analysis, or applicant-level explanations. This is important because loan decisions are consequential and users may need to understand why a prediction was produced.

No fairness or subgroup evaluation. The project does not evaluate whether prediction errors differ across demographic or other applicant groups. Since automated lending decisions can have unequal effects, fairness analysis would be necessary before considering practical deployment.

No external or temporal validation. The models are evaluated only on the supplied dataset. There is no separate external dataset or later-period dataset to test whether the learned relationships remain valid when applicant populations or economic conditions change. Therefore, the reported performance should not be interpreted as evidence of real-world deployment robustness.

Overall limitation assessment. The project provides a sound comparative machine-learning experiment, but its findings should be treated as an academic benchmark rather than a production-ready lending system. The strongest next step would be to use a larger representative dataset, apply consistent cross-validation to every model, tune hyperparameters, evaluate threshold-dependent and cost-sensitive performance, and add explainability and fairness analysis.